Siddhi Jha
A-3

ASSIGNMENT - 02

Problem Statement :
Write a C program for sparse matrix realization & operations on it - Transpose, Fast Transpose.

Objective :
1) To study the concept of sparse matrix, how it is stored & displayed
2) To understand the implementation of sparse matrix operations - simple Transpose, Fast Transpose.

Platform :
- 64-bit open source Linux or its derivatives.
- Open source C progg. tool like gcc/Eclipse Editor.

Pseudo Code :
Write pseudo code for input matrix to compact form, Simple Transpose & Fast Transpose.

Matrix to Compact form :

```
input rows, cols
input matrix [rows][cols]

count ← 0

for i ← 0 to rows - 1
    for j ← 0 to cols - 1
        if matrix [i][j] ≠ 0 then
            count ← count + 1
        end if
    end for
```

```
end for

compact[0][0] ← rows
compact[0][1] ← cols
compact[0][2] ← count

k ← 1
for i ← 0 to rows - 1
    for j ← 0 to cols-1
        if matrix[i][j] ≠ 0 then
            compact[k][0] ← i
            compact[k][1] ← j
            compact[k][2] ← matrix[i][j]
            k ← k+1
        end if
    end for
end for

display compact matrix
STOP
```

* Simple Transpose of Sparse Matrix :

```
START
input compact matrix
transpose[0][0] ← compact[0][1]
transpose[0][1] ← compact[0][0]
transpose[0][2] ← compact[0][2]
k ← 1
for col ← 0 to compact[0][1] - 1
    for i ← 1 to compact[0][2]
        if compact[i][1] = col then
```

```
            transpose [k][0] ← compact [i][1]
            transpose [k][1] ← compact [i][0]
            transpose [k][2] ← compact [i][2]
            k ← k + 1
        end if
    end for
end for
display transpose matrix
STOP
```

* Fast transpose of Sparse Matrix

```
input compact matrix
numCols ← compact [0][1]
numTerms ← compact [0][2]
fastTranspose [0][0] ← compact [0][1]
fastTranspose [0][1] ← compact[0][0]
fastTranspose [0][2] ← numTerms

if numTerms > 0 then
  for i ← 0 to numCols - 1
      rowTerms [i] ← 0
  end for
startingPos[0] ← 1
For i ← 1 to numCols-1
    startingPos [i] ←
    startingPos [i - 1] + rowTerms [i - 1]
End for
For i ← 1 to numTerms
    j ← startingPos[compact [i][1]]
    fastTranspose[j][0] ← compact[i][1]
    fastTranspose [j][1] ← compact [i][0]
```

fastTranspose[j][2] ← compact[i][2]

startingPos[compact[i][1]] ←
startingPos[compact[i][1]] + 1

END FOR
END IF
Display fastTranspose matrix
STOP

Time Complexity of Simple Transpose:
In simple Transpose, algorithm scans all columns for each column it scans all non-zero elements

For col = 0 to n-1
    for i = 1 to t
where, n = no. of columns
t = no. of non-zero elements (terms)
∴ T(n) = n x t
Time complexity : O(n x t)

Time complexity of Fast Transpose
Fast transpose performs three linear passes :
1) Count elements in each column → O(t)
2) Computes starting positions → O(n)
3) Place transposed elements → O(t)
Total : O(t) + O(n) + O(t) = O(n + 2t)
Ignoring constants,
Time complexity : O(n + t)

Conclusion:

Thus, implemented sparse matrix operations assignment. This system is able to perform different operations on sparse matrices such as simple & fast transpose & their time complexities.

FAQs

a) What is a sparse matrix? List its applications.

Ans: A sparse matrix is a matrix in which most of elements are 0. The number of non-zero elements is much smaller than the total number of elements.

Eg: $\begin{bmatrix} 0 & 0 & 5 \\ 0 & 8 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ Here, only 2 elements are non-zero out of 9 elements, so it is a sparse matrix.

Applications:

1) Graph representation (Adjacency Matrix)
2) Scientific & Engineering computations.
3) Image Processing
4) Finite Element Analysis (FEA).
5) Machine learning & Data Mining.
6) Network analysis
7) Database Systems.
8) Computer Graphics.

b) Representation of Sparse Matrix.

1) Triplet (3-Tuple) Representation

A sparse matrix is stored using three values:

- Row Number
- Column Number
- Value.

Eg $\begin{bmatrix} 0 & 0 & 5 \\ 0 & 8 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ triplet form :

Row	Col	Value
3	3	2
0	2	5
1	1	8

First row stores :
- Total Rows = 3
- Total Columns = 3
- Total non-zero elements = 2

2) Linked list Representation
Each node stores :
- Rows
- columns
- Value
- Next Pointer

This saves memory & allows dynamic insertion/deletion

c) Addition of 2 Sparse Matrices :

M1

Row	col	Value
4	5	6
0	3	5
1	3	45
2	3	4
3	2	45
4	1	2

M2

Row	Col	Value
4	5	6
0	3	7
0	4	6
1	4	4
2	1	8
3	2	45
4	4	21

Addition Result (M1 + M2)
Combine elements having same row & column

Row	Col	Value
4	5	9
0	3	12
0	4	6
1	3	45
1	4	4
2	1	8
2	3	4
3	2	90
4	1	2
4	4	21

Simple Transpose of M1

Original M1:

Row	col	Value
4	5	6
0	3	5
1	3	45
2	3	4
3	2	45
4	1	2

Transpose (Row ↔ column)

Row	Col	Value
5	4	6
1	4	2
2	3	45
3	0	5
3	1	8
3	2	4
4	1	45

Result for simple & fast Transpose will be same but their time complexities are different.

23/2/22

FOR EDUCATIONAL USE

Assignment 2

```
#include <stdio.h>
```

[redacted]

```
Void createSparse(int mat[10][10], int rows, int cols, int sparse[100][3]);
Void displaySparse(int sparse[100][3]);
Void addSparse(int s1[100][3], int s2[100][3], int result[100][3]);
Void transposeSparse(int sparse[100][3], int transpose[100][3]);

Int main()
{
  Int r1, c1, r2, c2;
  Int mat1[10][10], mat2[10][10];
  Int sparse1[100][3], sparse2[100][3], result[100][3], trans[100][3];
  Int I, j, choice;

  Printf("Enter rows and columns of Matrix 1: ");
  Scanf("%d%d", &r1, &c1);

  Printf("Enter elements of Matrix 1:\n");
  For(I = 0; I < r1; i++)
    For(j = 0; j < c1; j++)
      Scanf("%d", &mat1[i][j]);

  createSparse(mat1, r1, c1, sparse1);

  printf("\nEnter rows and columns of Matrix 2: ");
  scanf("%d%d", &r2, &c2);
```

```
printf("Enter elements of Matrix 2:\n");
for(I = 0; I < r2; i++)
  for(j = 0; j < c2; j++)
    scanf("%d", &mat2[i][j]);

createSparse(mat2, r2, c2, sparse2);

do
{
  Printf("\n===== MENU =====\n");
  Printf("1. Display Compact Matrices\n");
  Printf("2. Add Sparse Matrices\n");
  Printf("3. Transpose Matrix 1\n");
  Printf("4. Transpose Matrix 2\n");
  Printf("5. Exit\n");
  Printf("Enter choice: ");
  Scanf("%d", &choice);

  Switch(choice)
  {
    Case 1:
      Printf("\nCompact Matrix 1:\n");
      displaySparse(sparse1);

      printf("\nCompact Matrix 2:\n");
      displaySparse(sparse2);
```

```
      break;

    case 2:
      if(r1 == r2 && c1 == c2)
      {
        addSparse(sparse1, sparse2, result);

        printf("\nAddition Result (Compact Form):\n");
        displaySparse(result);
      }
      Else
      {
        Printf("\nAddition not possible.\n");
      }
      Break;

    Case 3:
      transposeSparse(sparse1, trans);

      printf("\nTranspose of Matrix 1:\n");
      displaySparse(trans);
      break;

    case 4:
      transposeSparse(sparse2, trans);
```

```
            printf("\nTranspose of Matrix 2:\n");
            displaySparse(trans);
            break;

        case 5:
            printf("Program Terminated.\n");
            break;

        default:
            printf("Invalid Choice!\n");
    }

  } while(choice != 5);

  Return 0;
}

Void createSparse(int mat[10][10], int rows, int cols, int sparse[100][3])
{
  Int I, j, k = 1;

  Sparse[0][0] = rows;
  Sparse[0][1] = cols;
  Sparse[0][2] = 0;

  For(I = 0; I < rows; i++)
```

```
  {
    For(j = 0; j < cols; j++)
    {
      If(mat[i][j] != 0)
      {
        Sparse[k][0] = I;
        Sparse[k][1] = j;
        Sparse[k][2] = mat[i][j];
        K++;
        Sparse[0][2]++;
      }
    }
  }
}

Void displaySparse(int sparse[100][3])
{
  Int I;

  Printf("Row\tCol\tValue\n");

  For(I = 0; I <= sparse[0][2]; i++)
  {
    Printf("%d\t%d\t%d\n",
        Sparse[i][0],
        Sparse[i][1],
```

```
            Sparse[i][2]);
    }
}

Void addSparse(int s1[100][3], int s2[100][3], int result[100][3])
{
    Int I = 1, j = 1, k = 1;

    Result[0][0] = s1[0][0];
    Result[0][1] = s1[0][1];

    While(I <= s1[0][2] && j <= s2[0][2])
    {
        If(s1[i][0] == s2[j][0] && s1[i][1] == s2[j][1])
        {
            Result[k][0] = s1[i][0];
            Result[k][1] = s1[i][1];
            Result[k][2] = s1[i][2] + s2[j][2];

            I++;
            J++;
            K++;
        }
        Else if((s1[i][0] < s2[j][0]) ||
            (s1[i][0] == s2[j][0] && s1[i][1] < s2[j][1]))
        {
```

```
            Result[k][0] = s1[i][0];
            Result[k][1] = s1[i][1];
            Result[k][2] = s1[i][2];

            I++;
            K++;
        }
        Else
        {
            Result[k][0] = s2[j][0];
            Result[k][1] = s2[j][1];
            Result[k][2] = s2[j][2];

            J++;
            K++;
        }
    }

    While(I <= s1[0][2])
    {
        Result[k][0] = s1[i][0];
        Result[k][1] = s1[i][1];
        Result[k][2] = s1[i][2];
        I++;
        K++;
    }
```

```
  While(j <= s2[0][2])
  {
    Result[k][0] = s2[j][0];
    Result[k][1] = s2[j][1];
    Result[k][2] = s2[j][2];
    J++;
    K++;
  }

  Result[0][2] = k – 1;
}

Void transposeSparse(int sparse[100][3], int transpose[100][3])
{
  Int I, j, k = 1;

  Transpose[0][0] = sparse[0][1];
  Transpose[0][1] = sparse[0][0];
  Transpose[0][2] = sparse[0][2];

  For(I = 0; I < sparse[0][1]; i++)
  {
    For(j = 1; j <= sparse[0][2]; j++)
    {
      If(sparse[j][1] == i)
```

```
      {
        Transpose[k][0] = sparse[j][1];
        Transpose[k][1] = sparse[j][0];
        Transpose[k][2] = sparse[j][2];
        K++;
      }
    }
  }
}
```